

C++ Módulo 03

Guia de Referência Bibliográfica

Herança, Sobrescrita e o Problema do Diamante

Escola 42

1. Introdução à Herança

A herança é um dos conceitos fundamentais da Programação Orientada a Objetos (POO). Ela permite que uma nova classe (chamada de **Classe Derivada**) adquira as propriedades e métodos de uma classe existente (chamada de **Classe Base**). O principal objetivo da herança é a reutilização de código e o estabelecimento de uma relação semântica do tipo "é um" (ex: Um ScavTrap é um ClapTrap).

O Modificador 'protected'

Até o módulo anterior, o encapsulamento era feito majoritariamente com `private` (acessível apenas pela própria classe) e `public` (acessível por qualquer um). A herança introduz um novo nível de acesso:

- **protected:** Variáveis e métodos declarados como protegidos são acessíveis pela própria classe e por todas as suas classes derivadas, mas continuam ocultos e inacessíveis para funções externas ao escopo das classes. No módulo 03, os atributos de `ClapTrap` (nome, vida, energia, dano) mudam de `private` para `protected`.

2. Ciclo de Vida: Construtores e Destrutores

Entender a ordem em que as classes são montadas e desmontadas na memória é crucial em C++.

Encadeamento de Construção

A construção ocorre de "dentro para fora" ou "do ancestral mais antigo para o descendente mais novo". Quando instanciamos um `FragTrap`, o compilador **obrigatoriamente** chama o construtor da classe base `ClapTrap` antes de executar o construtor do `FragTrap`.

Encadeamento de Destruição

A destruição segue a regra LIFO (Last In, First Out). A destruição ocorre na ordem inversa da construção: primeiro o destrutor de FragTrap é chamado, e depois o destrutor de ClapTrap limpa a base da memória.

3. Sobrescrita de Métodos (Overriding)

Classes derivadas podem redefinir o comportamento de métodos herdados da classe base para fornecer uma funcionalidade específica. No exercício 01 e 02, sobrescrevemos o método `attack()`.

```
// Em ScavTrap.cpp, o ataque é redefinido:
void ScavTrap::attack(const std::string& target) {
    if (_hitPoints <= 0 || _energyPoints <= 0)
        return;
    _energyPoints--;
    std::cout << "ScavTrap " << _name << " VIOLENTLY ATTACKS "
                << target << "!" << std::endl;
}
```

Quando chamamos `attack()` num objeto `ScavTrap`, o compilador ignora a versão do `ClapTrap` e utiliza esta nova implementação.

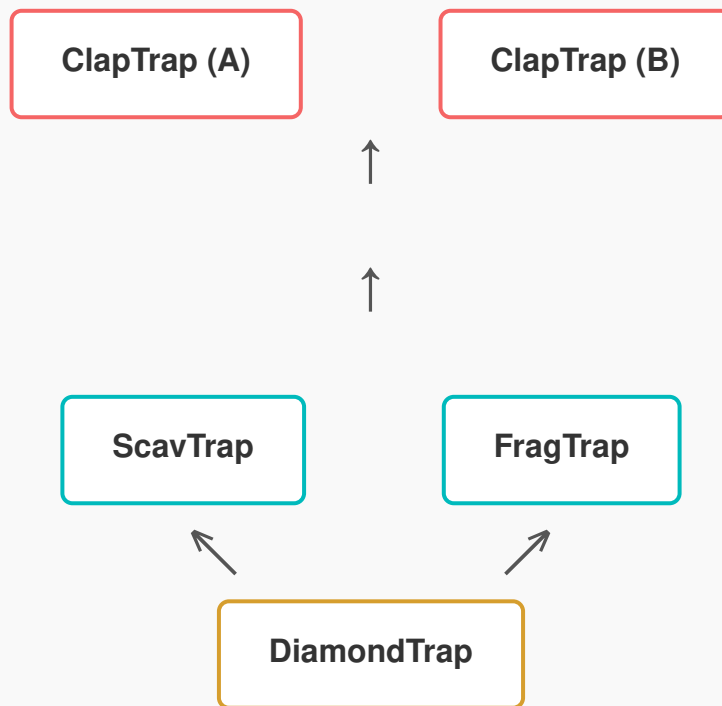
4. Herança Múltipla e o Problema do Diamante

O C++ permite a **Herança Múltipla**, onde uma classe pode ter mais de uma classe base. O Exercício 03 introduz o `DiamondTrap`, que herda simultaneamente de `ScavTrap` e `FragTrap`.

Como ambos, `ScavTrap` e `FragTrap`, herdam de `ClapTrap`, nós nos deparamos com o infame **Problema do Diamante**. O compilador criará duas cópias independentes de `ClapTrap` na

memória para um único DiamondTrap. Isso gera ambiguidade (Qual `_hitPoints` usar? O do lado ScavTrap ou do FragTrap?).

A Estrutura Problemática (Duplicação na Memória)



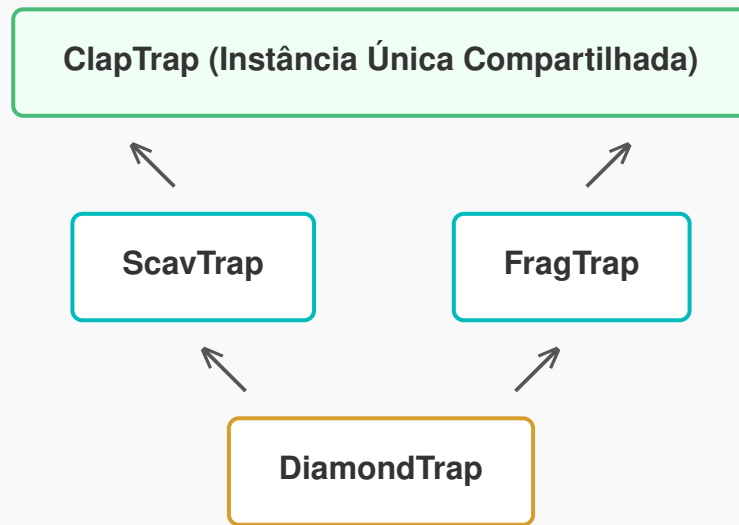
A Solução: Herança Virtual

Para resolver essa ambiguidade e evitar o desperdício de memória, usamos a **herança virtual**. Ao declarar que ScavTrap e FragTrap herdam de ClapTrap de forma virtual, instruímos o compilador a instanciar ClapTrap **apenas uma vez**, e essa instância única será compartilhada por toda a árvore genealógica.

```
// ScavTrap.hpp
class ScavTrap : virtual public ClapTrap { ... };

// FragTrap.hpp
class FragTrap : virtual public ClapTrap { ... };
```

Memória Compartilhada com Herança Virtual



5. Name Shadowing & Flags de Compilação

No DiamondTrap, você foi instruído a criar um atributo privado `std::string _name`. Como a classe base ClapTrap também possui um `_name`, a variável da classe derivada "sombreia" (esconde) a variável da classe base.

-Wshadow: A flag de compilação `-Wshadow` serve exatamente para alertar o programador sobre essa prática, pois muitas vezes é fruto de um erro ou desatenção e pode gerar bugs difíceis de rastrear. Neste projeto específico (ex03), o shadowing é intencional.

Para resolver conflitos e acessar explicitamente uma variável ou método da classe base quando sofreu shadowing, usamos o operador de resolução de escopo `::`.

```
void DiamondTrap::whoAmI() {  
    std::cout << "I am " << this->_name  
                << " and my ClapTrap name is "  
                << ClapTrap::_name << std::endl;  
}
```